# AI Agent Discovery

Presenter: Nic Williams (nic@infoblox.com)

Co-Proponents (problem space): Laurent Ciavaglia laurent.ciavaglia@nokia.com, Arashmid Akhavain arashmid.akhavain@huawei.com, Hesham Moussa hesham.moussa@huawei.com, Roland Schott Roland.Schott@telekom.de, Zaheduzzaman Sarker zahed.sarker.ietf@gmail.com, Jim Mozley jmozley@infoblox.com, Daniel King daniel@olddog.co.uk, Adrian Farrel adrian@olddog.co.uk



Internal

# PROBLEM SPACE

There is a functional layer **before** discovery which is the gate of which agents should be discoverable. This may be enforced bidirectionally by an entity owner, and the entity they wish to discover may use different policies.

This discovery is as simple as asking "Find me an entity to talk to." However, there is a lot of detail underlying this question including:

- What functionality do we want an entity to provide for us?
- How do I communicate with that entity?
- Is the information about the entity trustable?

There is a functional layer **after** discovery that determines which instance of a class of entity *should* be used. This process may examine location, reachability, load, and operational status. Those pieces of information have to be examinable, but they do not form part of the core requirements of discovery.

Out of scope: How do I register my entity (with option for attestation), trust, responsibility, capability and negotiation, tasks management, agent-to-agent communications


Internal

# SHOULD | SHOULD NOT

www.example.com is the label for a web site and is used to find it.

- DNS is not used to communicate with the web site, nor get the web site content.

index._agents.example.com could be the label to find an organization's AI agents (in this case, a registry).

- DNS is not used to communicate with the index of agents, nor get the details of all the agents.

DNS used as stable beachhead to find well-known agents, and well-known registries of agents.


Internal

# SHOULD | SHOULD NOT

The DNS used to express full agent capabilities.

Enumerate the DNS to search for a best-suited agent for a task or all agents within a domain, OR all domains with agents supporting X capability.

Be used solely as a trust, verification, or identity-related mechanism.

Create a new RRType, or modify RDATA logic to send all records in a zone.


Internal

# DNS APPROACH TO SUBSECTION OF PROBLEM

Discovery status:

Known service & domain: DNS (eg copilot.microsoft.com)

Known service | domain: index (example.com, what are all your agents?)

NOR Known service | domain: registry / search-engine (what organizations support agents that can do X?)

Record status:

_agents.example.com

SVCB w/ key params for agent/application-specific metadata & model card URI

TXT fallback for DNS that don't support SVCB

Problem statements:

https://datatracker.ietf.org/doc/draft-mozley-aidiscovery/

https://github.com/danielkinguk/discovery/blob/main/README.md#problem-statement-and-work-scope



Internal

# "SUPPORTERS"

Existing similar DNS-based AI discovery I-Ds (beyond ours):

https://datatracker.ietf.org/doc/draft-nemethi-aid-agent-identity-discovery/

https://datatracker.ietf.org/doc/draft-cui-dns-native-agent-naming-resolution/

https://datatracker.ietf.org/doc/draft-liu-cats-dns-service-discovery/

CN NIC side meeting presentation (waiting for link to slides / drafts)

Complementary drafts:

"Use TXT RR to list payment services" https://datatracker.ietf.org/doc/draft-jeftovic-x402-dns-discovery/

"The index to consolidate AI DNS records" https://ad.jaime.win/draft-jimenez-ad-agent-directory

"Combine index and DNS" https://www.ietf.org/archive/id/draft-narajala-ans-00.html



Internal

# QUESTIONS


Internal

# COMMON OBJECTIONS

Why not /well-known/?

- Out of band PKI to validate endpoint association
- Knowing HTTPS origin first + querying DNS to get there
- Caching semantics for CDN / intermediaries may fragment depending on metadata support... outages become single points of failure for entire discovery
- Fragment observability and policy controls between whichever options each HTTP origin implements, standardized view into what was discovered and by whom more difficult (eg we know chat._example.com from IP xxx.xxx.xxx.xxx is an AI workload)

… There will need to be registry / HTTP to solve for the use cases when you don't know the exact agent, our argument is most times enterprises through trust relationships will know the agent for the task


Internal